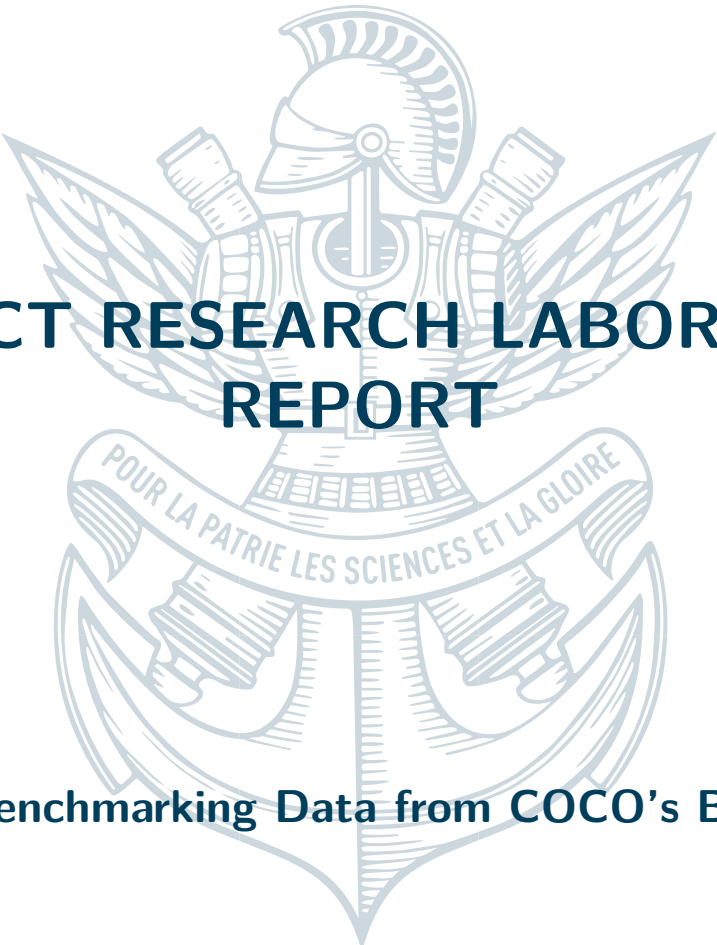




PROJECT RESEARCH LABORATORY REPORT

Analyzing Benchmarking Data from COCO's BBOB Suites

December 31, 2025

Elsa Felts

Supervised by Professor Dimo Brockhoff



Abstract

This project develops a methodology to compare more than 250 derivative-free optimization algorithms evaluated in the COCO (Comparing Continuous Optimizers) platform across multiple years, dimensions, and test suites. Using Expected Running Time (ERT) as the performance measure, the analysis identifies the best algorithms for each dimension and shows that no single method dominates the noiseless BBOB (Black-Box Optimization Benchmarking) suite, while a small group consistently leads in the noisy case. In the bi-objective suite, one algorithm clearly outperforms all others across dimensions. By aggregating the results and visualizing dominance patterns, the project provides a compact view of the large COCO archive and reveals stable trends that help guide algorithm selection for real-world black-box optimization tasks.

CONTENTS

1	Introduction	3
2	Background and Theoretical Framework	3
2.1	Black-Box Optimization: Setting and Motivation	3
2.1.1	Search space and problem instances.	4
2.1.2	Why benchmarking is essential.	4
2.1.3	The COCO platform	4
2.2	Families of Algorithms in the COCO Archive	4
3	Performance Indicators	5
3.1	Target Precision Levels	5
3.2	Success Criterion	5
3.3	Expected Running Time (ERT)	6
4	Different test Suites	6
4.1	The BBOB Noiseless Test Suite	6
4.1.1	General Characteristics of the BBOB noiseless test suite	7
4.1.2	The Five Function Families	7
4.2	The BBOB Noisy Test Suite	7
4.2.1	General Principles of the Noisy Suite	8
4.2.2	Noise Models	8
4.3	The BBOB Bi-objective Test Suite	8
5	Implementation	9
5.1	Hierarchy of the Benchmark Data and COCO tools	9
5.2	Method 1: Absolute Best Algorithm Extraction	9
5.2.1	Extracting the Best Algorithms per Year	9
5.2.2	Aggregating Results Across All Years	10
5.2.3	Variant of Method 1: Best-Algorithm Extraction for a Fixed Target	10
5.3	Method 2: Near-Optimal Algorithms; the Factor Method	11
5.3.1	Motivation	11
5.3.2	Defining the Factor Method	11
5.3.3	Implementation of the Factor-Based Method	11
5.3.4	Algorithmic Procedure	12

6	Results	12
6.1	BBOB (Noiseless)	13
6.1.1	Method 1: Absolute Best Algorithm	13
6.1.2	Method 1 (variation): Absolute Best Algorithm	14
6.1.3	Method 2: Factor-Based Near-Optimality	14
6.2	BBOB noisy	15
6.2.1	Method 1 : Absolute Best Algorithm	15
6.2.2	Method 1 (variation: Absolute Best Algorithm)	16
6.2.3	Method 2: Factor-Based Near-Optimality	16
6.3	BBOB Bi-Objective (biobj)	17
6.3.1	Method 1 : Absolute Best Algorithm	17
6.3.2	Method 1 (variation: Absolute Best Algorithm)	18
6.3.3	Method 2: Factor-Based Near-Optimality	19
6.4	Summary of observation	21
7	Conclusion	21

1 INTRODUCTION

The RandOpt team studies optimization problems where the objective function is unknown and can only be evaluated through costly simulations or physical models. Such black-box settings appear frequently in industry, where gradients are unavailable and each evaluation can require expensive computations. Efficient optimization algorithms are therefore crucial.

To support this need, RandOpt develops the COCO platform (COmparing Continuous Optimizers), a standardized environment for testing and comparing algorithms under reproducible conditions. COCO is widely used in research and industry because it provides consistent benchmark functions, reliable performance measures, and uniform reporting.

Over the years, COCO has accumulated results from hundreds of algorithms, making it difficult to extract clear insights or identify consistently strong performers. Raw outputs are informative but too large to interpret directly.

This project addresses this issue by developing a methodology to analyze and compare more than 250 derivative-free algorithms across the BBOB (Black-Box Optimization Benchmarking) test suites. The objective is to identify the best algorithms for each dimension and target precision, and to understand how performance evolves with problem difficulty. By aggregating multi-year data and producing standardized visualizations, the project condenses the extensive COCO archive into an interpretable set of results that reveals stable, high-performing algorithms.

2 BACKGROUND AND THEORETICAL FRAMEWORK

2.1 BLACK-BOX OPTIMIZATION: SETTING AND MOTIVATION

Black-box optimization refers to a class of optimization problems in which the internal structure of the objective function is unknown to the algorithm (see below). The only permitted interaction is through function evaluations: the optimizer proposes a candidate solution $x \in S$, where S denotes the search space of the algorithm (see below), and receives the numerical value $f(x)$, with no access to gradients, analytic expressions, or structural information such as smoothness, conditioning, separability, or multimodality. For this reason, black-box

methods are also referred to as derivative-free optimizers. Their behavior must rely exclusively on the sequence of observed evaluations

$$(f(x_1), f(x_2), \dots, f(x_t)),$$

hence the term “black box.”

2.1.1 • SEARCH SPACE AND PROBLEM INSTANCES.

Formally, black-box optimization problems considered in the BBOB benchmark are defined over a search space

$$S = S_1 \times \dots \times S_n \subset \mathbb{R}^n,$$

(see the part 1 of [2]). This domain is chosen to be large enough to include the essential structural features of the different functions, such as valleys, ridges, and multimodal patterns, while ensuring comparability across algorithms. A candidate solution is any point $x \in S$. The corresponding objective function is written

$$f_\theta : S \rightarrow \mathbb{R},$$

where θ denotes the instance of the problem. Although the mathematical form of the function remains the same across instances, its “location and orientation” change through transformations such as shifting, rotating, or rescaling. This instance mechanism is crucial: it prevents algorithms from exploiting accidental symmetries or coordinate alignments and enables statistically meaningful performance assessment. Benchmarking in BBOB typically averages results over 15 independent instances to remove accidental lucky behavior.

During optimization, the algorithm generates a sequence of points $(x_t)_{t \geq 1}$, evaluates them, and updates its internal state based on the observed values $(f_\theta(x_t))_{t \geq 1}$. The entire search process is therefore driven by the information gained after each evaluation.

2.1.2 • WHY BENCHMARKING IS ESSENTIAL.

Benchmarking is essential in black-box optimization because algorithms have access only to function evaluations. Their performance can therefore be assessed only by observing how many evaluations they need to reach given target precisions. A framework like COCO provides a controlled and reproducible setting for this assessment: it defines standardized function instances, common target values, and a unified performance measure based on evaluation counts. This ensures that differences between solvers reflect genuine algorithmic behavior rather than experimental artifacts. Benchmarking thus makes it possible to draw meaningful statements such as “Algorithm A reaches the same precision with fewer evaluations than Algorithm B” [2] under strictly identical conditions.

2.1.3 • THE COCO PLATFORM

The Black-Box Optimization Benchmarking (BBOB) test suites are designed to capture a wide range of optimization challenges while maintaining interpretability and comparability. The COCO platform provides a collection of parameterized functions reflecting different structures, multiple problem dimensions $n \in \{2, 3, 5, 10, 20, 40\}$, enabling scalability studies, a set of transformed instances per function, ensuring statistical robustness, and standardized performance targets and indicators, most notably the Expected Running Time (ERT).

The combination of these elements makes BBOB a powerful and widely adopted benchmark for analyzing the behavior, strengths, and limitations of derivative-free optimizers. The COCO platform provides a unified and controlled environment that allows researchers to evaluate algorithms systematically and to isolate the impact of different factors such as conditioning, separability, noise, or dimensionality.

2.2 FAMILIES OF ALGORITHMS IN THE COCO ARCHIVE

The COCO benchmark archive contains results from a wide variety of derivative-free optimization algorithms. Although the focus of this report is on extracting and analyzing performance trends, a brief overview of the algorithmic landscape helps contextualize the results and highlights the diversity of approaches being compared.

- **Evolutionary Strategies (ES).** A large part of the archive consists of CMA-ES variants such as CMA-ES, IPOP-CMA-ES, BIPOP-CMA-ES, and numerous algorithmic extensions incorporating adaptation schemes or restart mechanisms.
- **Quasi-Newton Methods.** Algorithms such as BFGS or L-BFGS rely on finite-difference approximations of gradients. These methods exploit local curvature information and can be highly efficient on smooth, moderately conditioned functions.
- **Trust-Region Algorithms.** Methods such as NEWUOA or TR-BFGS iteratively build and refine local models of the objective within a region of trust. These approaches often perform well on smooth problems where accurate local models can be constructed.
- **Model-Based and Surrogate Methods.** This class includes Bayesian optimization, SLSQP, and other algorithms that construct surrogate models (e.g., Gaussian processes, quadratic models) to guide the search efficiently.
- **Direct Search / Pattern Search Methods.** Examples include Nelder–Mead, Powell’s method, and related pattern-search variants. These algorithms explore the search space by evaluating structured sets of points without explicitly modeling gradients or curvature.

Although these families have different strengths and weaknesses, COCO provides a uniform testing environment that enables meaningful comparisons across all of them. This diversity also explains why, in many cases, no single algorithm dominates the entire benchmark suite.

3 PERFORMANCE INDICATORS

To compare optimization algorithms in a fair and reproducible way, the COCO platform relies on a small number of standard performance indicators. These indicators are defined so that results can be compared across functions, dimensions, and instances. In this project, the most important one is the Expected Running Time (ERT), which is the basis of all analyses presented later.

3.1 TARGET PRECISION LEVELS

The BBOB framework defines a set of 51 target precision levels, ranging from coarse accuracy (large thresholds) to very fine accuracy (extremely small thresholds). Targets are predefined thresholds on the function value and correspond to different levels of accuracy. These targets allow the evaluation of how an algorithm behaves at different stages of the search: some algorithms reach easy targets quickly but struggle to achieve high precision, while others may progress slowly at first but excel at more difficult targets.

In this project, we use a selected subset of these targets that covers the full range of difficulties. This reduces computational overhead while still providing a representative picture of algorithmic performance.

3.2 SUCCESS CRITERION

A run is considered successful if it reaches the target precision before the evaluation budget is exhausted.

3.3 EXPECTED RUNNING TIME (ERT)

The Expected Running Time measures how many function evaluations an algorithm needs, on average, to reach a given target value of the objective function. More precisely, the ERT depends only on the target precision, the function, and the dimension. The choice of instance does not change the definition of ERT itself; rather, it affects the empirical estimate of ERT, because we approximate it using a finite number of runs over several randomized instances. This averaging over 15 instances makes the resulting ERT values statistically meaningful and less sensitive to randomness.

Expected Running Time (ERT)

For a given target precision Δf , the ERT is defined over the 15 BBOB instances of a function as:

$$\text{ERT}(\Delta f) = p_s \cdot \text{RT}_{\text{success}} + (1 - p_s) \cdot \text{RT}_{\text{unsuccess}} = \frac{\sum_{i=1}^{15} \text{RT}_i}{\#\text{successes}}.$$

Where:

- RT_i is the number of function evaluations performed in run i before the target precision Δf is reached.
- If the target is not reached in run i , then RT_i is set to the full evaluation budget of that run.
- $\#\text{successes}$ is the number of runs (out of 15) that reach the target.

To obtain the empirical expression from the probabilistic one, we estimate

$$p_s = \frac{\#\text{successes}}{15},$$

where 15 is the total number of independent runs. By definition, $\text{RT}_{\text{success}}$ is the average number of evaluations over successful runs, whereas $\text{RT}_{\text{unsuccess}}$ corresponds to the maximal evaluation budget (since the target was not reached).

Special case: If no run reaches the target, then

$$\text{ERT}(\Delta f) = +\infty.$$

4 DIFFERENT TEST SUITES

4.1 THE BBOB NOISELESS TEST SUITE

The BBOB (Black-Box Optimization Benchmarking) consists of 24 real-valued functions, covering a wide range of structural properties such as separability, conditioning, multimodality, and ruggedness.

Each BBOB function admits an infinite number of randomized instances. In practice, the COCO benchmarking experiments use 15 instances [1]: these are specific realizations generated through shifts, rotations, and other transformations applied to the base function. To avoid overfitting algorithms to a fixed set of cases, the first 5 instances are kept constant across benchmark years, while the remaining 10 instances are changed periodically. This design preserves reproducibility and enables a statistically meaningful comparison of algorithms,

while ensuring that algorithms cannot exploit coordinate alignments or special symmetries.

The noiseless suite evaluates algorithmic performance under ideal circumstances, i.e. without stochastic perturbations in the objective value. This allows us to distinguish structural difficulty (e.g., narrow valleys, ill-conditioning, deceptive landscapes) from noise-induced difficulty, which is addressed in the next section.

4.1.1 • GENERAL CHARACTERISTICS OF THE BBOB NOISELESS TEST SUITE

The 24 functions are partitioned into five conceptual families, each representing a particular optimization challenge. They are evaluated across dimensions: $n \in \{2, 3, 5, 10, 20, 40\}$, making it possible to study the scalability of algorithms. Almost all functions share the same search domain $[-5, 5]^n$, which ensures fairness and comparability across families and algorithms. Below, we provide a high-level overview of the five families.

4.1.2 • THE FIVE FUNCTION FAMILIES

The 24 BBOB functions are grouped into five families, each representing a different optimisation scenario. The first family (f1–f5) contains simple, separable unimodal functions whose landscapes point directly toward the optimum; they mainly test basic exploitation and coordinate-wise search. Families 2 and 3 (f6–f14) introduce variable interaction and increasing levels of conditioning, creating curved or narrow valleys; these functions evaluate an algorithm’s ability to adapt its search when the landscape is no longer aligned with the coordinate axes. The fourth family (f15–f19) consists of multimodal functions that still possess an overall global structure, testing whether an algorithm can avoid local traps while following the general trend of improvement. The final family (f20–f24) contains highly multimodal and deceptive functions without any global guidance, making them the most challenging; they test an algorithm’s exploratory capacity and robustness. This organization enables the benchmark to highlight how algorithms that perform similarly on simpler tasks may differ substantially when faced with structural complexity. It also helps identify strengths and weaknesses: an algorithm excelling on separable and moderately conditioned functions may still fail on deceptive multi-modal ones.

Below we provide a visual of what can be found on the COCO platform to visualize the problem.

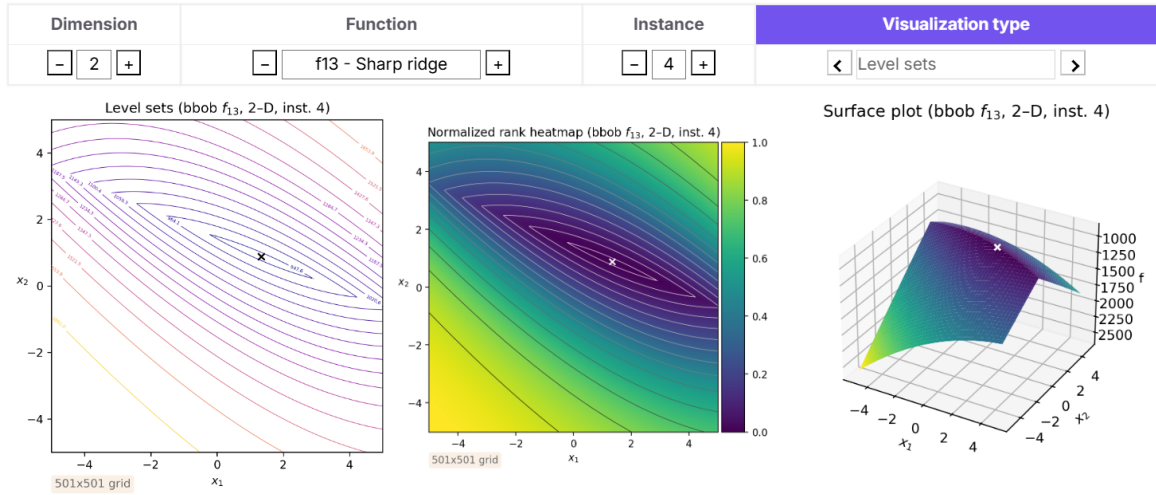


Figure 1: Landscape visualization of BBOB f_{13} (Sharp Ridge), 2-D – Instance 4: level sets, normalized rank heatmap, and 3D surface.

4.2 THE BBOB NOISY TEST SUITE

In the BBOB noisy suite, every function evaluation includes a random disturbance. This setup reflects many real situations where the value of a function cannot be measured exactly and is inherently stochastic. For

example, when measurements rely on sensors, simulations, or sampling procedures. The BBOB noisy test suite explicitly models such uncertainty and provides a controlled environment to study the robustness and reliability of algorithms under noise.

The noisy suite combines the same underlying function structures as in the noiseless suite (separable, unimodal, multimodal, etc.), but injects stochastic perturbations of varying intensity and distribution. By comparing performance on both noiseless and noisy versions, one can isolate the effect of noise and understand which algorithms degrade and which fail catastrophically when function evaluations become unreliable.

4.2.1 • GENERAL PRINCIPLES OF THE NOISY SUITE

In the noisy suite, the perturbation mechanism depends on the type of noise used. Only the Cauchy noise model is additive: in this case, the observed function value is given by

$$f(x) = f_{\text{base}}(x) + \varepsilon.$$

For the Gaussian and Uniform noise models, the perturbation is instead multiplicative, meaning that the observed value scales with the underlying function value:

$$f(x) = f_{\text{base}}(x) (1 + \varepsilon).$$

This distinction is essential: under multiplicative noise, the amplitude of the perturbation automatically becomes smaller as the search approaches the optimum, thus increasing the signal-to-noise ratio and allowing optimization algorithms to continue making progress. In contrast, additive Cauchy noise introduces heavy-tailed perturbations that remain large regardless of how close the algorithm is to the optimum, making the problem substantially more challenging. As in the noiseless suite, each function comes with 15 randomized instances that include independent shifts and rotations. Noise is generated independently for every evaluation.

Studying noise is essential for understanding the practical stability of optimization algorithms.

4.2.2 • NOISE MODELS

The noisy BBOB suite extends the original functions by adding stochastic perturbations. It includes three noise families: Gaussian (f101–f103), uniform (f104–f106), and heavy-tailed noise (f107–f109), each provided at both moderate and severe levels. Gaussian noise introduces light-tailed fluctuations that roughly preserve the landscape, uniform noise creates bounded but unpredictable perturbations, and heavy-tailed noise occasionally produces large outliers. Moderate noise keeps the global structure mostly visible, while severe noise can dominate the function values and obscure the underlying trend, making progress substantially harder. All noisy functions share the same base landscapes as the noiseless suite, so any additional difficulty comes purely from randomness. This separation helps identify which algorithms are robust to uncertainty and which algorithms only perform well in ideal deterministic conditions. Overall, this setup makes it possible to assess how robust an algorithm is when evaluations become uncertain or inconsistent.

4.3 THE BBOB BI-OBJECTIVE TEST SUITE

In many real-world problems, it is not enough to optimize a single objective: one often needs to balance two conflicting goals at the same time. Typical examples include minimizing cost while maximizing performance, or reducing weight while increasing stability. Such settings define bi-objective optimization problems.

Pareto Dominance

A solution x dominates a solution y if it is at least as good in both objectives and strictly better in at least one of them.

Pareto Optimality. A solution is Pareto-optimal if no other solution dominates it.

Pareto Front. The set of all Pareto-optimal objective vectors forms the *Pareto front*, which represents the best achievable trade-offs between the two objectives. Depending on the underlying functions, this front may be straight, curved, convex, concave, or even disconnected.

The BBOB bi-objective test suite is designed to evaluate algorithms in this setting. Instead of a single objective function $f(x)$, each test problem consists of a pair of functions $(f_1(x), f_2(x))$, and the goal is no longer to identify a single “best” point, but to approximate the entire Pareto front as accurately as possible. The suite combines pairs of single-objective BBOB functions to generate a wide range of bi-objective problems. Depending on the chosen pair, the resulting trade-off landscape may be smooth or rugged, convex or highly multimodal, and may exhibit varying degrees of conflict between objectives. This diversity ensures that algorithms are tested across simple, moderately difficult, and highly challenging bi-objective scenarios.

Note that the set of available target precisions in the bi-objective suite coincides with the standard BBOB targets except that the finest precision level 10^{-8} is not available; the smallest target used is 10^{-5} .

5 IMPLEMENTATION

We will implement two complementary methods for identifying top-performing algorithms. The first selects the absolute best algorithm for each (dimension, function, target) triplet and counts how often each algorithm appears as the unique winner. The second defines “best” more broadly by including all algorithms whose ERT lies within a chosen performance factor of the minimum. This produces a small group of near-optimal optimizers that can be compared more meaningfully.

5.1 HIERARCHY OF THE BENCHMARK DATA AND COCO TOOLS

All data used in this project come from the official COCO archives. The BBOB benchmark organizes algorithm evaluations by problem dimension and function. For each (dimension, function) pair, performance is defined for multiple target precisions Δf , for which COCO provides Expected Running Time (ERT) values. For this reason, the analysis in this project follows the hierarchy:

dimension $\rightarrow$ function $\rightarrow$ target precision $\rightarrow$ algorithm.

For each triple (dimension, function, target), all algorithms are compared on the basis of their Expected Running Time (ERT). If an algorithm never reaches the target precision value Δf , these cases are removed before selecting the best-performing algorithm for that (dimension, function, target) triple.

Although BBOB defines 51 canonical target precisions, here we are only using a selected subset of target that spans the full difficulty range (coarse to fine accuracy). This reduces the computational overhead of the code, while still capturing the essential differences in algorithmic behavior.

5.2 METHOD 1: ABSOLUTE BEST ALGORITHM EXTRACTION

5.2.1 • EXTRACTING THE BEST ALGORITHMS PER YEAR

For each BBOB year, we identify the best-performing algorithm separately. This allows us to later compare results across years and test whether certain algorithms consistently dominate the benchmark.

The following pseudo code summarizes the per-year extraction:

Pseudo code

```

load the data for a specific year
initialise a counter (algorithm -> number of wins)

for each dimension:
    for each function:
        for each selected target precision:
            compute ERT for all algorithms
            discard algorithms with ERT = +infinity
            select the algorithm with smallest ERT
            increment its counter

store the list of best algorithms for this year
export results to CSV

```

5.2.2 • AGGREGATING RESULTS ACROSS ALL YEARS

After extracting the yearly best algorithms, we combine all results into a single dataset in order to identify the global best algorithm across all years. For each (dimension, function, target) triple, we select the algorithm with the smallest ERT among all years. We purposely chose not to loop over all the years at once, and break it down into each year, as the runtime for gathering and processing the data of a single year was already quite long. As an output, we produce a list of algorithms that perform best not only within a single year, but consistently across the entire BBOB archive.

5.2.3 • VARIANT OF METHOD 1: BEST-ALGORITHM EXTRACTION FOR A FIXED TARGET

We also implemented a simplified interactive variant of Method 1 that allows the user to focus on a single target precision. In many practical situations, one may want to identify the best-performing algorithm specifically at a given target Δf , rather than across the entire set of targets. To support this, the user is prompted to enter the desired target value.

Since Method 1 already produces a CSV file containing, for every $(d, f, \Delta f)$ triplet, we simply filter this CSV to retain only the rows associated with the chosen target. We then aggregate the winners across all functions for each dimension, yielding a dimension-wise ranking of algorithms at the selected precision level.

Pseudocode: Best Algorithm at User-Selected Target

```

# ask user to choose a target df
target = user_input()
# Step 1: load global-best CSV from Method 1
table = load_csv("global_best_algorithms.csv")
# Step 2: filter to the chosen target
filtered = table where df == target
# Step 3: count winners per dimension
initialize counter[d][A] = 0
for each row (d, f, A) in filtered:
    counter[d][A] += 1
# Step 4: export results
export counter[d][A] to CSV

```

5.3 METHOD 2: NEAR-OPTIMAL ALGORITHMS; THE FACTOR METHOD

In addition to identifying the single best algorithm for each (dimension,function,target) triplet, we introduce a second method that selects all algorithms whose performance is sufficiently close to the optimal one. This factor method provides a more robust assessment of algorithmic performance, particularly in situations where many optimizers achieve comparable ERT values.

5.3.1 • MOTIVATION

Relying only on the absolute best algorithm can be misleading in large benchmarks such as BBOB, where more than 250 optimizers often achieve very similar ERT values. A strict winner-only approach highlights only the smallest ERT, even when several algorithms are essentially indistinguishable in practice, causing many robust near-optimal methods to be overlooked.

Small variations in ERT rarely correspond to meaningful performance differences. Noise in the evaluations, or even minor numerical effects in the noiseless suite, can create accidental gaps that make one algorithm appear strictly better than another, despite almost identical behavior.

The factor method addresses this issue by grouping together all algorithms whose ERT lies within a chosen multiplicative factor of the optimal one. This ensures that near-equivalent algorithms are treated equally, rather than being separated by arbitrary numerical differences.

5.3.2 • DEFINING THE FACTOR METHOD

Definition of the Factor Method

Let $\mathcal{A}$ denote the set of all algorithms evaluated on the BBOB benchmark for all given years. For each problem instance $(d, f, \Delta f)$ defined by the dimension d , the function f , and the target precision Δf , every algorithm $A \in \mathcal{A}$ is associated with an Expected Running Time value $\text{ERT}_A(d, f, \Delta f)$.

The factor method generalizes the strict winner approach by selecting not only the algorithm with the smallest ERT, but all algorithms that perform sufficiently close to the best one. We define the minimal ERT on this instance as

$$\text{ERT}_{\min}(d, f, \Delta f) = \min_{A \in \mathcal{A}} \{\text{ERT}_A(d, f, \Delta f)\}.$$

Given a performance factor $\alpha > 1$, an algorithm $A \in \mathcal{A}$ is considered near-optimal if

$$\text{ERT}_A(d, f, \Delta f) \leq \alpha \cdot \text{ERT}_{\min}(d, f, \Delta f).$$

For $\alpha = 1$, the method reduces to selecting only the strict winner. Larger values of α yield increasingly inclusive groups of high-performing algorithms.

5.3.3 • IMPLEMENTATION OF THE FACTOR-BASED METHOD

We first reuse the reference values obtained in Method 1, namely the globally minimal Expected Running Time

$$\text{ERT}_{\min}(d, f, \Delta f)$$

computed for each combination $(d, f, \Delta f)$ across all BBOB years.

We then perform a second pass over the full multi-year dataset. For every tuple (year, $d, f, \Delta f$), we retrieve the ERT values of all algorithms and identify those satisfying

$$\text{ERT}_A(\text{year}, d, f, \Delta f) \leq \alpha \cdot \text{ERT}_{\min}(d, f, \Delta f).$$

Each such algorithm is marked as near-optimal and recorded together with its associated metadata (year, dimension, function, target). Finally, for each dimension d , we aggregate these occurrences and count how many times each algorithm A appears in the near-optimal set across all years, functions, and target precisions. The resulting frequency tables indicate which algorithms consistently perform within a factor α of the global best ERT.

5.3.4 • ALGORITHMIC PROCEDURE

Before executing the procedure, the user is prompted to choose the value of the performance factor α . Rather than hard coding α in advance, the implementation allows interactive selection of this coefficient, after which the search for near-optimal algorithms is carried out based on the chosen value.

Pseudocode for the Factor Method

```
# Step 1: load best ERTs from Method 1
load CSV from Method 1 containing ERT_min(d, f, target)
# ask user to choose performance factor a > 1
a = user_input()
# Step 2: scan full multi-year dataset
initialize list NEAR_OPTIMAL_ROWS = []
for each year Y:
    load data for year Y
    for each dimension d:
        for each function f:
            for each target precision target:
                get ERT[A] for all algorithms A in this (Y, d, f, target)
                discard algorithms with ERT[A] = +infinity
                retrieve ERT_min(d, f, target) from Method 1 table
                for each algorithm A:
                    if ERT[A] <= a * ERT_min(d, f, target):
                        append (A, Y, d, f, target) to NEAR_OPTIMAL_ROWS

# Step 3: output all near-optimal occurrences
write NEAR_OPTIMAL_ROWS to CSV file
# Step 4: aggregate by dimension
initialize counter[d][A] = 0
for each row (A, Y, d, f, target) in NEAR_OPTIMAL_ROWS:
    counter[d][A] += 1
# output aggregation results per dimension
export counter[d][A] to CSV
```

6 RESULTS

We now present the results obtained for the three benchmark suites considered in this project: BBOB (noiseless), BBOB-noisy, and BBOB-biobj. For each suite, we evaluate algorithm performance using both the strict winner-based approach (Method 1) and the factor-based near-optimal approach (Method 2). For BBOB, we also report the interactive variant of Method 1, which focuses on a user-selected target precision. Each set of results includes summary tables and visualization graphs.

6.1 BBOB (NOISELESS)

6.1.1 • METHOD 1: ABSOLUTE BEST ALGORITHM

The table in figure 2 reports, for each algorithm, the total number of wins achieved across all dimensions, all functions, and all target precisions. This quantity therefore measures overall dominance across the entire BBOB suite.

Overall frequency of being best by algorithm and dimension (top 10):

dimension	2	3	5	10	20	40	Total
best_algorithm							
SLSQP+lq-CMA-ES_Hansen	6	4	0	12	4	11	37
BIPOP-aCMA-STEP_loshchilov	2	5	7	6	4	11	35
SLSQP-scipy-2019_Varelas	1	0	5	5	9	14	34
HMLS_pal	16	9	7	1	0	0	33
HE-ES_Glasmachers	0	5	15	6	5	0	31
LSfminbnd_posik	5	5	5	5	5	5	30
BIPOP-saACM-k_loshchilov	0	0	0	9	11	9	29
SHADE-LM-POP4-to-10_Okulewicz	6	5	5	5	5	0	26
OQNLP_pal	0	7	12	2	3	0	24
fminunc_pal	4	9	5	0	5	0	23

How many best algorithms do you want? 1

Requested 1 best algorithms.

Returning 6 unique algorithms (reached rank 1).

List of selected algorithms:

- CMAES-APOP-KMA_Nguyen
- HE-ES_Glasmachers
- HMLS_pal
- SLSQP+lq-CMA-ES_Hansen
- SLSQP-scipy-2019_Varelas
- lq-CMA-ES_Hansen

Figure 2: BBOB: Total Number of Wins Achieved Across all Dimensions

Figure 3: List of the 6 Best Algorithms

Figure 4 shows the table of best algorithms, aggregated across all years of the BBOB suite. This visualization makes it possible to compare algorithmic dominance separately for each dimension. While a few algorithms appear more frequently as strict winners for some dimensions, the margins are modest, with most competing methods showing similar win counts. This indicates that dominance exists but is not strong, and several algorithms perform at comparable levels. Algorithms with identical win counts are tied; alphabetical order is used only for display.

dimension	2	3	5	10	20	40
rank						
1	(HMLS_pal, 16)	(lq-CMA-ES_Hansen, 12)	(HE-ES_Glasmachers, 15)	(SLSQP+lq-CMA-ES_Hansen, 12)	(CMAES-APOP-KMA_Nguyen, 12)	(SLSQP-scipy-2019_Varelas, 14)
2	(DE-BFGS_vogliss, 9)	(HMLS_pal, 9)	(OQNLP_pal, 12)	(BIPOP-saACM-k_loshchilov, 9)	(BIPOP-saACM-k_loshchilov, 11)	(BIPOP-aCMA-STEP_loshchilov, 11)
3	(NELDERDOERR_doerr, 8)	(fminunc_pal, 9)	(BIPOP-aCMA-STEP_loshchilov, 7)	(BIRMIN_Kudela, 8)	(NIPOPacMA_loshchilov, 10)	(SLSQP+lq-CMA-ES_Hansen, 11)
4	(DTS-CMA-ES_Pitra, 7)	(OQNLP_pal, 7)	(HMLS_pal, 7)	(BIPOP-aCMA-STEP_loshchilov, 6)	(L-BFGS-B-scipy-2019_Varelas, 9)	(NEWUOA_ros, 10)
5	(SHADE-LM-POP4-to-10_Okulewicz, 6)	(NELDERDOERR_doerr, 6)	(PSA-CMA-ES_Nishida, 7)	(FULLNEWUOA_ros, 6)	(SLSQP-scipy-2019_Varelas, 9)	(BIPOP-saACM-k_loshchilov, 9)

Figure 4: BBOB Suite: Ranking of Algorithms per Dimension for Method 1, Showing (Algorithm, Number of Wins).

We also implemented an interactive variant (see Figure 3) where the user specifies a number $N_{\min}$ of desired algorithms. The system progressively collects algorithms from the first ranking row, then from the second row, and so on, until at least $N_{\min}$ unique algorithms are obtained.

For instance, if the user requests $N_{\min} = 1$, the method returns all algorithms that appear as rank 1 in at least one dimension. For larger $N_{\min}$, additional ranking rows are incorporated until the threshold is met.

This tool provides a simple way to extract a shortlist of competitive candidates independent of dimension.

6.1.2 • METHOD 1 (VARIATION): ABSOLUTE BEST ALGORITHM

Available targets: [1.e-08 1.e-05 1.e-03 1.e-02 1.e-01]

Select a target precision (e.g., 1e-8 or 1e-5): 1e-8

Ranking table built for target = 1e-08

dimension	2	3	5	10	20	40
rank						
1	(DE-BFGS_vogliss, 2)	(lq-CMA-ES_Hansen, 3)	(HE-ES_Glasmachers, 3)	(BIPOP-saACM-k_loschilov, 3)	(BIPOP-saACM-k_loschilov, 6)	(BIPOP-saACM-k_loschilov, 5)
2	(DTS-CMA-ES_Pitra, 2)	(GLOBAL_pal, 2)	(BIPOP-aCMA-STEP_loschilov, 2)	(HCMA_loschilov, 3)	(CMAES-APOP-KMA_Nguyen, 3)	(BIPOP-aCMA-STEP_loschilov, 3)
3	(HMLS_pal, 2)	(NELDER_hansen, 2)	(GLOBAL_pal, 2)	(BIPOPsaACM_loschilov, 2)	(NIPOPaCMA_loschilov, 3)	(CMAES-APOP-KMA_Nguyen, 2)

Figure 5: BBOB Suite Method 1: Ranking of Algorithms for a Given Target per Dimension, Showing (Algorithm, Number of Wins).

Figure 5 shows, at the fixed target $\Delta f = 10^{-8}$, the strict winner method produces only very small win counts, with most algorithms winning between two and three function–dimension combinations. Since BBOB contains 24 functions, a maximum count of 3 indicates that no algorithm dominates the benchmark at this target precision. Instead, performance appears highly fragmented, with several algorithms winning only a small number of instances and no clear pattern of superiority emerging.

This behavior suggests that Method 1, when restricted to a single target precision, does not provide a sufficiently strong signal to identify a consistent best algorithm. The small win counts reflect the fact that at high precision levels, many algorithms achieve comparable performance, leading to numerous ties. This further motivates the use of Method 2: the factor-based approach captures groups of algorithms whose performance is practically equivalent, and is therefore better suited to detecting robust high-performing sets under specific target precision settings.

6.1.3 • METHOD 2: FACTOR-BASED NEAR-OPTIMALITY

Method 2 allows an algorithm to be counted as a near-optimal performer whenever its ERT is within a chosen factor α of the minimum observed ERT. This produces a richer and more stable picture of performance than strict winner-based counting and avoids overemphasizing tiny numerical differences that have no practical significance.

rank	dim 2	dim 3	dim 5	dim 10	dim 20	dim 40
1	SLSQP+lq-CMA-ES_Hansen (52)	lq-CMA-ES_Hansen (54)	lq-CMA-ES_Hansen (59)	SLSQP-11-scipy_Hansen (39)	SLSQP+lq-CMA-ES_Hansen (41)	SLSQP+lq-CMA-ES_Hansen (43)
2	DIRECT-REV_Kudela (51)	SLSQP+lq-CMA-ES_Hansen (41)	SLSQP+lq-CMA-ES_Hansen (47)	SLSQP+lq-CMA-ES_Hansen (38)	IPOPsaACM_loschilov (36)	SLSQP-11-scipy_Hansen (34)
3	NELDERDOERR_doerr (45)	SLSQP-11-scipy_Hansen (37)	SLSQP-11-scipy_Hansen (46)	BIPOPsaACM_loschilov (35)	lq-CMA-ES_Hansen (30)	NEWUOA_ros (30)
4	SLSQP-11-scipy_Hansen (45)	DIRECT-REV_Kudela (36)	SLSQP-scipy-2019_Varelas (36)	lq-CMA-ES_Hansen (33)	SLSQP-11-scipy_Hansen (29)	adm-CMA-ES_Gissler (30)
5	lq-CMA-ES_Hansen (36)	SHADE-LM-POP4-to-10_Okulewicz (34)	CMA-ES-2019_Hansen (27)	IPOPsaACM_loschilov (27)	NIPOPaCMA_loschilov (27)	SLSQP-scipy-2019_Varelas (28)
6	DTS-CMA-ES_Pitra (34)	SLSQP-scipy-2019_Varelas (33)	SHADE-LM_Okulewicz (26)	adm-CMA-ES_Gissler (24)	PSA-CMA-ESwRS_Nishida (27)	NIPOPaCMA_loschilov (27)

Figure 6: Ranking of Algorithms per Dimension for the Factor Method, (Algorithm, Number of Wins)

Figure 6 reports the factor-based ranking of algorithms separately for each dimension $d \in \{2, 3, 5, 10, 20, 40\}$.

Each entry gives the number of times an algorithm appears within the near-optimal factor threshold for that dimension. The factor-based results reveal a set of consistently competitive algorithms.

Figure 7 aggregates the near-optimal counts over all functions, dimensions, and Δf targets.

In contrast to Method 1, where strict wins rarely exceeded 30–40, the factor-based method produces much larger counts: the top algorithms achieve over 200 near-optimal hits. This demonstrates that many algorithms perform similarly to the best ERT, even if they are not strictly the absolute minimizer. This provides a clearer view of overall robustness than strict win counting, while avoiding over-emphasis on small numerical differences.

Overall frequency of being best (aggregated over ALL dimensions):

	algorithm	count
119	SLSQP+lq-CMA-ES_Hansen	262
139	lq-CMA-ES_Hansen	230
120	SLSQP-11-scipy_Hansen	230
121	SLSQP-scipy-2019_Varelas	177
18	BFGS-P-StPt_Blelly	140
65	DIRECT-REV_Kudela	127
52	DE-BFGS_vogliss	123
126	a-CMA-ES_Gissler	122
129	adm-CMA-ES_Gissler	121
97	NEWUOA_ros	119

Figure 7: Total Number of Wins Achieved Across all Dimensions using the the Factor Method

6.2 BBOB NOISY

6.2.1 • METHOD 1 : ABSOLUTE BEST ALGORITHM

For the BBOB noisy test suite, we produce the same type of tables as in the noiseless case.

Overall frequency of being best by algorithm and dimension (top 10):

dimension	2	3	5	10	20	40	Total
best_algorithm							
IPOPsaACM_loshchilov_noisy	12	29	42	38	55	0	176
IPOP-ACTCMA-ES_ros_noisy	20	22	14	13	11	53	133
1koma4mirser_brockhoff_noisy	11	17	18	18	16	37	117
IPOP-CMA-ES_ros_noisy	10	5	12	13	20	25	85
SNES_schaul_noisy	4	8	9	9	3	4	37
PSAaSmC-CMA-ES_Nishida_bbob-noisy	2	8	4	8	14	0	36
FULLNEWUOA_ros_noisy	12	7	7	8	1	0	35
BIPOP-CMA-ES_hansen_noisy	4	3	6	9	4	2	28
SNOBFIT_huyer_noisy	9	9	7	2	0	0	27
PSAaLmD-CMA-ES_Nishida_bbob-noisy	3	9	10	0	0	0	22

Figure 8: BBOB noisy: Detailed Total Number of Wins Achieved Across all Dimensions

Figure ?? shows the per-dimension ranking for Method 1. For each dimension d , the table lists the top-ranked algorithms along with the number of strict wins they achieve.

Some trends emerge from this table. We have the noisy variants of CMA-ES that remain dominant across all dimensions. But their dominance is not striking, as the counts remain in the same range.

Like in the BBOB test suite, we also implemented the interactive variant of Method 1, in which the user specifies a number $N_{\min}$ of desired algorithms. For instance, when requesting $N_{\min} = 1$, only two algorithms appear as rank 1 across all dimensions: IPOP-ACTCMA-ES_ros_noisy and IPOPsaACM_loshchilov_noisy. This indicates a clear pair of dominant performers at the strict-winner level. What we will want to investigate is whether, the dominance of these 2 algorithms is inflated with the factor method (see method 2).

dimension	2	3	5	10	20	40
rank						
1	(IPOP-ACTCMA-ES_ros_noisy, 20)	(IPOPsaACM_loschilov_noisy, 29)	(IPOPsaACM_loschilov_noisy, 42)	(IPOPsaACM_loschilov_noisy, 38)	(IPOPsaACM_loschilov_noisy, 55)	(IPOP-ACTCMA-ES_ros_noisy, 53)
2	(FULLNEUWOA_ros_noisy, 12)	(IPOP-ACTCMA-ES_ros_noisy, 22)	(1komma4mirser_brockhoff_noisy, 18)	(1komma4mirser_brockhoff_noisy, 18)	(IPOP-CMA-ES_ros_noisy, 20)	(1komma4mirser_brockhoff_noisy, 37)
3	(IPOPsaACM_loschilov_noisy, 12)	(1komma4mirser_brockhoff_noisy, 17)	(IPOP-ACTCMA-ES_ros_noisy, 14)	(IPOP-ACTCMA-ES_ros_noisy, 13)	(1komma4mirser_brockhoff_noisy, 16)	(IPOP-CMA-ES_ros_noisy, 25)
4	(1komma4mirser_brockhoff_noisy, 11)	(PSAaLmD-CMA-ES_Nishida_bbob-noisy, 9)	(IPOP-CMA-ES_ros_noisy, 12)	(IPOP-CMA-ES_ros_noisy, 13)	(PSAaSmC-CMA-ES_Nishida_bbob-noisy, 14)	(CMAEGS_finck_noisy, 5)
5	(AMALGAM_bosman_noisy, 11)	(SNOBFIT_huyer_noisy, 9)	(PSAaLmD-CMA-ES_Nishida_bbob-noisy, 10)	(BIPOP-CMA-ES_hansen_noisy, 9)	(IPOP-ACTCMA-ES_ros_noisy, 11)	(SNES_schau_noisy, 4)
6	(IPOP-CMA-ES_ros_noisy, 10)	(PSAaLmC-CMA-ES_Nishida_bbob-noisy, 8)	(SNES_schau_noisy, 9)	(SNES_schau_noisy, 9)	(RCGA_tran_noisy, 5)	(BIPOP-CMA-ES_hansen_noisy, 2)

Figure 9: Ranking of Algorithms per Dimension for the Method 1, showing (Algorithm, Number of Wins)

6.2.2 • METHOD 1 (VARIATION: ABSOLUTE BEST ALGORITHM)

Ranking table built for target = 1e-08

dimension	2	3	5	10	20	40
rank						
1	(IPOP-ACTCMA-ES_ros_noisy, 5)	(IPOPsaACM_loschilov_noisy, 6)	(IPOPsaACM_loschilov_noisy, 8)	(IPOPsaACM_loschilov_noisy, 7)	(IPOPsaACM_loschilov_noisy, 10)	(IPOP-ACTCMA-ES_ros_noisy, 13)
2	(FULLNEUWOA_ros_noisy, 4)	(IPOP-ACTCMA-ES_ros_noisy, 5)	(1komma4mirser_brockhoff_noisy, 5)	(1komma4mirser_brockhoff_noisy, 4)	(IPOP-ACTCMA-ES_ros_noisy, 6)	(1komma4mirser_brockhoff_noisy, 7)
3	(1komma4mirser_brockhoff_noisy, 3)	(1komma4mirser_brockhoff_noisy, 4)	(IPOP-ACTCMA-ES_ros_noisy, 3)	(IPOP-ACTCMA-ES_ros_noisy, 4)	(1komma4mirser_brockhoff_noisy, 3)	(IPOP-CMA-ES_ros_noisy, 2)

Figure 10: BBOB noisy: Ranking of Algorithms per Dimension for a specific Target Precision (1e-08) (Algorithm, Number of Wins)

Figures 10 report the strict-winner counts when restricting the analysis to a single target precision (here $\Delta f = 10^{-8}$). For each dimension, the table shows the top algorithms and their win counts. For this specific target precision, the same 2 algorithms appear. This accentuates the dominance of these 2 algorithms. The results thus motivate the use of Method 2, whose factor-based definition provides a more reliable measure of robustness under noisy evaluations.

6.2.3 • METHOD 2: FACTOR-BASED NEAR-OPTIMALITY

As expected from the Method 1, across the two representations of the data: the per-dimension tuple table (figure 10), and the aggregated win counts (figure 11) a much sharper dominance pattern appears. The factor method thus inflated the count for the top 2 algorithms identified previously. In method 1, the top 2 algorithms were sharing the counts for the absolute best algorithm position. However, this factor method focuses on the performance of an algorithm rather than it being the best, this leads to greater counts, as they do not need to share the first position anymore.

The strong count separation is also visible dimension-wise: the same leading methods consistently occupy the top ranks for each dimension, producing a stable and coherent ordering independent of dimensionality. The aggregated near-optimal counts confirm the strong dominance pattern already visible in the dimension-wise tables. This behavior contrasts with the noiseless results, where performance was more fragmented, and dominance was difficult to attribute to any small group of optimizers. In the noisy setting, however, the convergence of evidence across tables, and totals, indicates that a small set of algorithms reliably outperforms the rest and can be identified with confidence.

rank	dim 2	dim 3	dim 5	dim 10	dim 20	dim 40
1	IPOPsaACM_ishchilov_noisy (78)	IPOPsaACM_ishchilov_noisy (101)	IPOPsaACM_ishchilov_noisy (97)	IPOPsaACM_ishchilov_noisy (101)	IPOPsaACM_ishchilov_noisy (115)	IPOP-ACTCMA-ES_ros_noisy (102)
2	IPOP-ACTCMA-ES_ros_noisy (68)	IPOP-ACTCMA-ES_ros_noisy (88)	IPOP-ACTCMA-ES_ros_noisy (80)	IPOP-ACTCMA-ES_ros_noisy (87)	IPOP-ACTCMA-ES_ros_noisy (97)	IPOP-CMA-ES_ros_noisy (69)
3	IPOP-CMA-ES_ros_noisy (53)	IPOP-CMA-ES_ros_noisy (50)	IPOP-CMA-ES_ros_noisy (53)	IPOP-CMA-ES_ros_noisy (51)	IPOP-CMA-ES_ros_noisy (51)	BIPOP-CMA-ES_hansen_noisy (64)
4	BIPOP-CMA-ES_hansen_noisy (45)	BIPOP-CMA-ES_hansen_noisy (47)	BIPOP-CMA-ES_hansen_noisy (44)	BIPOP-CMA-ES_hansen_noisy (44)	BIPOP-CMA-ES_hansen_noisy (42)	1komma4mirser_brockhoff_noisy (41)
5	SNES_schaul_noisy (40)	PSAaLmC-CMA-ES_Nishida_bbob-noisy (41)	PSAaLmC-CMA-ES_Nishida_bbob-noisy (37)	PSAaSMD-CMA-ES_Nishida_bbob-noisy (39)	1komma4mirser_brockhoff_noisy (42)	IPOP-SEP-CMA-ES_ros_noisy (35)
6	1komma4mirser_brockhoff_noisy (39)	1komma4mirser_brockhoff_noisy (38)	PSAaLmD-CMA-ES_Nishida_bbob-noisy (31)	PSAaSmC-CMA-ES_Nishida_bbob-noisy (37)	PSAaSmC-CMA-ES_Nishida_bbob-noisy (32)	SNES_schaul_noisy (10)

Figure 10: BBOB Noisy: Ranking of Algorithms per Dimension for the Factor Method, Showing (Algorithm, Number of Wins)

Overall frequency of being best (aggregated over ALL dimensions):

	algorithm	count
20	IPOP-ACTCMA-ES_ros_noisy	522
23	IPOPsaACM_ishchilov_noisy	492
21	IPOP-CMA-ES_ros_noisy	327
13	BIPOP-CMA-ES_hansen_noisy	286
6	1komma4mirser_brockhoff_noisy	220
31	PSAaSmC-CMA-ES_Nishida_bbob-noisy	136
29	PSAaLmC-CMA-ES_Nishida_bbob-noisy	134
22	IPOP-SEP-CMA-ES_ros_noisy	126
37	SNES_schaul_noisy	120
5	1komma4mir_brockhoff_noisy	117

Figure 11: BBOB Noisy, Factor method: Frequency Count of the Top 10 Algorithms

6.3 BBOB BI-OBJECTIVE (BIOBJ)

6.3.1 • METHOD 1 : ABSOLUTE BEST ALGORITHM

Some trends emerge from this table (figure 12). We observe a small group of algorithms that consistently rise to the top for every dimension. In the bi-objective case the dominance pattern is even more pronounced: the leading algorithm not only appears in the first rank across almost all dimensions, but also has a substantially higher win count than the competitors; in one case, the count can differ by over 100. The separation becomes even clearer when we aggregate over all dimensions (figure 13). A difference of 300 in the count separates the first to the second algorithm. From both tables we can conclude that the best algorithm occupies the leaderboard in a much more stable and uniform manner. The identification of top performers is more straightforward than in both the noiseless and noisy single-objective cases. The next sections will examine whether this sharp dominance persists for a given factor, and if the dominance becomes inflated when using the factor-based criterion of Method 2.

dimension	2	3	5	10	20	40
rank						
1	(SPEA2-platypus_Brockhoff_bbob-biobj, 53)	(HMO-CMA-ES_Loshchilov_bbob-biobj, 82)	(HMO-CMA-ES_Loshchilov_bbob-biobj, 99)	(HMO-CMA-ES_Loshchilov_bbob-biobj, 125)	(HMO-CMA-ES_Loshchilov_bbob-biobj, 131)	(GDE3-platypus_Brockhoff_bbob-biobj, 109)
2	(UP-MO-CMA-ES_Krause_bbob-biobj, 51)	(SPEA2-platypus_Brockhoff_bbob-biobj, 38)	(UP-MO-CMA-ES_Krause_bbob-biobj, 38)	(UP-MO-CMA-ES_Krause_bbob-biobj, 20)	(TPB_Tanabe, 21)	(RM-MEDA_Auger_bbob-biobj, 77)
3	(HMO-CMA-ES_Loshchilov_bbob-biobj, 50)	(UP-MO-CMA-ES_Krause_bbob-biobj, 33)	(SPEA2-platypus_Brockhoff_bbob-biobj, 30)	(TPB_Tanabe, 19)	(UP-MO-CMA-ES_Krause_bbob-biobj, 21)	(SPEA2-platypus_Brockhoff_bbob-biobj, 26)
4	(K-RVEA_Tanabe, 26)	(K-RVEA_Tanabe, 20)	(TPB_Tanabe, 19)	platypus_Brockhoff_bbob-biobj, 18)	(GDE3-platypus_Brockhoff_bbob-biobj, 19)	(MOEAD-platypus_Brockhoff_bbob-biobj, 9)
5	(IBEA-platypus_Brockhoff_bbob-biobj, 16)	(TPB_Tanabe, 18)	(GDE3-platypus_Brockhoff_bbob-biobj, 17)	(SPEA2-platypus_Brockhoff_bbob-biobj, 18)	(SPEA2-platypus_Brockhoff_bbob-biobj, 14)	(SMS-EMOA-DE_Auger_bbob-biobj, 1)

Figure 12: BBOB Bi-Objective: Table showing the Absolute Best Top 5 algorithms per Dimension

Overall frequency of being best by algorithm and dimension (top 10):

dimension	2	3	5	10	20	40	Total
best_algorithm							
HMO-CMA-ES_Loshchilov_bbob-biobj	50	82	99	125	131	0	487
GDE3-platypus_Brockhoff_bbob-biobj	11	15	17	17	19	109	188
SPEA2-platypus_Brockhoff_bbob-biobj	53	38	30	18	14	26	179
UP-MO-CMA-ES_Krause_bbob-biobj	51	33	38	20	21	0	163
RM-MEDA_Auger_bbob-biobj	14	14	3	2	1	77	111
TPB_Tanabe	13	18	19	19	21	0	90
IBEA-platypus_Brockhoff_bbob-biobj	16	16	14	18	11	0	75
K-RVEA_Tanabe	26	20	10	1	1	0	58
MOTPE_Tanabe	15	13	10	3	0	0	41
COMO-316_dufosse_bbob-biobj	1	4	13	15	7	0	40

Figure 13: BBOB Bi-Objective: Detailed Count per Dimension of the Top 10 Best Algorithms

6.3.2 • METHOD 1 (VARIATION: ABSOLUTE BEST ALGORITHM)

Note that the target precision $1e-8$ doesn't exist anymore for the BBOB biobj test suite. Across all examined target precisions (10^{-3} and 10^{-5}), the fixed-target tables (figure 14) display a remarkably stable pattern: the same small group of algorithms systematically occupies the top ranks for every dimension and every target. This recurrence is fully consistent with the dominance already observed in the aggregated Method 1 results. The bi-objective landscape produces almost identical rankings regardless of the chosen precision level. This stability reinforces the conclusion that a clearly defined subset of algorithms consistently outperforms the others on the bi-objective testbed, making the selection of top-performing methods straightforward.

Ranking table built for target = 0.001

dimension	2	3	5	10	20	40
rank						
1	(HMO-CMA-ES_Loshchilov_bbob-biobj, 14)	(HMO-CMA-ES_Loshchilov_bbob-biobj, 24)	(HMO-CMA-ES_Loshchilov_bbob-biobj, 32)	(HMO-CMA-ES_Loshchilov_bbob-biobj, 43)	(HMO-CMA-ES_Loshchilov_bbob-biobj, 45)	(RM-MEDA_Auger_bbob-biobj, 24)
2	(RM-MEDA_Auger_bbob-biobj, 14)	(RM-MEDA_Auger_bbob-biobj, 6)	(COMO-316_dufosse_bbob-biobj, 7)	(COMO-316_dufosse_bbob-biobj, 8)	(COMO-316_dufosse_bbob-biobj, 3)	(GDE3-platypus_Brockhoff_bbob-biobj, 13)
3	(GDE3-platypus_Brockhoff_bbob-biobj, 7)	(GDE3-platypus_Brockhoff_bbob-biobj, 5)	(UP-MO-CMA-ES_Krause_bbob-biobj, 5)	(GDE3-platypus_Brockhoff_bbob-biobj, 2)	(UP-MO-CMA-ES_Krause_bbob-biobj, 3)	(SPEA2-platypus_Brockhoff_bbob-biobj, 4)

Ranking table built for target = 1e-05

dimension	2	3	5	10	20	40
rank						
1	(UP-MO-CMA-ES_Krause_bbob-biobj, 28)	(HMO-CMA-ES_Loshchilov_bbob-biobj, 26)	(UP-MO-CMA-ES_Krause_bbob-biobj, 24)	(HMO-CMA-ES_Loshchilov_bbob-biobj, 21)	(HMO-CMA-ES_Loshchilov_bbob-biobj, 22)	(GDE3-platypus_Brockhoff_bbob-biobj, 14)
2	(HMO-CMA-ES_Loshchilov_bbob-biobj, 16)	(UP-MO-CMA-ES_Krause_bbob-biobj, 22)	(HMO-CMA-ES_Loshchilov_bbob-biobj, 15)	(UP-MO-CMA-ES_Krause_bbob-biobj, 9)	(UP-MO-CMA-ES_Krause_bbob-biobj, 11)	(MOEAD-platypus_Brockhoff_bbob-biobj, 2)
3	(SPEA2-platypus_Brockhoff_bbob-biobj, 4)	(SPEA2-platypus_Brockhoff_bbob-biobj, 3)	(COMO-100_dufosse_bbob-biobj, 3)	(COMO-316_dufosse_bbob-biobj, 3)	(COMO-100_dufosse_bbob-biobj, 3)	(RM-MEDA_Auger_bbob-biobj, 1)

Figure 14: BBOB Bi-Objective: Top 3 algorithms for 2 Different Target Precisions: 1e-03 and 1e-05

6.3.3 • METHOD 2: FACTOR-BASED NEAR-OPTIMALITY

Across the two representations of the bi-objective results: the dimension-wise tuple table (figure 15), and the aggregated win counts (figure 16), a very clear dominance structure emerges, even more pronounced than in the noisy single-objective suite. The leading algorithm occupies an exceptionally large count of all best performances, and the gap to the next competitors is substantial. This concentration is visible across every dimension: the top 4 algorithms repeatedly occupy the leading positions from $d = 2$ to $d = 40$ (note that the the algorithm HMO-CMA-ES-Loshchilov-bbob-biobj doesn't appear for $d=40$ simply because it is not tested on this dimension), suggesting a high degree of robustness to dimensionality. Unlike the noiseless suites, where the performance landscape was more diffuse and several algorithms remained statistically close, the bi-objective results show a strong and stable hierarchy. The convergence of evidence across all three data views indicates that a small group of algorithms consistently dominates the bi-objective testbed, making the selection of high-performing candidates significantly more straightforward.

rank	dim 2	dim 3	dim 5	dim 10	dim 20	dim 40
1	HMO-CMA-ES_Loshchilov_bbob-biobj (125)	HMO-CMA-ES_Loshchilov_bbob-biobj (135)	HMO-CMA-ES_Loshchilov_bbob-biobj (141)	HMO-CMA-ES_Loshchilov_bbob-biobj (152)	HMO-CMA-ES_Loshchilov_bbob-biobj (159)	GDE3-platypus_Brockhoff_bbob-biobj (130)
2	MO-DIRECT-HV-Rank_Wong_bbob-biobj (74)	SPEA2-platypus_Brockhoff_bbob-biobj (55)	UP-MO-CMA-ES_Krause_bbob-biobj (48)	TPB_Tanabe (45)	TPB_Tanabe (42)	RM-MEDA_Auger_bbob-biobj (96)
3	UP-MO-CMA-ES_Krause_bbob-biobj (68)	K-RVEA_Tanabe (53)	TPB_Tanabe (41)	UP-MO-CMA-ES_Krause_bbob-biobj (38)	UP-MO-CMA-ES_Krause_bbob-biobj (34)	SPEA2-platypus_Brockhoff_bbob-biobj (62)
4	DEMO_Tusar_bbob-biobj (63)	UP-MO-CMA-ES_Krause_bbob-biobj (49)	SPEA2-platypus_Brockhoff_bbob-biobj (39)	GDE3-platypus_Brockhoff_bbob-biobj (38)	GDE3-platypus_Brockhoff_bbob-biobj (31)	MOEAD-platypus_Brockhoff_bbob-biobj (24)
5	SPEA2-platypus_Brockhoff_bbob-biobj (61)	TPB_Tanabe (46)	K-RVEA_Tanabe (37)	COMO-316_dufosse_bbob-biobj (32)	SPEA2-platypus_Brockhoff_bbob-biobj (23)	SMS-EMOA-DE_Auger_bbob-biobj (1)

Figure 15: BBOB Bi-Objective: Ranking of Algorithms per Dimension for the Factor method, Showing (Algorithm, Number of Wins)

Overall frequency of being best (aggregated over ALL dimensions):

	algorithm	count
9	HMO-CMA-ES_Loshchilov_bbob-biobj	712
8	GDE3-platypus_Brockhoff_bbob-biobj	311
30	SPEA2-platypus_Brockhoff_bbob-biobj	265
32	UP-MO-CMA-ES_Krause_bbob-biobj	237
27	RM-MEDA_Auger_bbob-biobj	216
31	TPB_Tanabe	211
11	K-RVEA_Tanabe	164
10	IBEA-platypus_Brockhoff_bbob-biobj	159
13	MO-CMA-ES-100-autoref_Dufosse_bbob-biobj	126
15	MO-DIRECT-HV-Rank_Wong_bbob-biobj	126

Figure 16: BBOB Bi-Objective, Factor method: Frequency Count of the Top 10 Algorithms

Like in the target selection in Method 1, I implemented a filtering over a specific target for every single test suite, (here the BBOB Biobj). This enables us to refine our search of the best algorithm, when we know the dimension of our problem and the target precision we would like to achieve.

Ranking table for target = 1e-05

dimension	2	3	5	10	20	40
rank						
1	(HMO-CMA-ES_Loshchilov_bbob-biobj, 42)	(HMO-CMA-ES_Loshchilov_bbob-biobj, 41)	(HMO-CMA-ES_Loshchilov_bbob-biobj, 31)	(HMO-CMA-ES_Loshchilov_bbob-biobj, 29)	(HMO-CMA-ES_Loshchilov_bbob-biobj, 32)	(GDE3-platypus_Brockhoff_bbob-biobj, 14)
2	(UP-MO-CMA-ES_Krause_bbob-biobj, 35)	(UP-MO-CMA-ES_Krause_bbob-biobj, 28)	(UP-MO-CMA-ES_Krause_bbob-biobj, 28)	(UP-MO-CMA-ES_Krause_bbob-biobj, 21)	(UP-MO-CMA-ES_Krause_bbob-biobj, 20)	(MOEAD-platypus_Brockhoff_bbob-biobj, 2)
3	(MO-CMA-ES-100-autoref_Dufosse_bbob-biobj, 19)	(MO-CMA-ES-100-autoref_Dufosse_bbob-biobj, 5)	(COMO-316_dufosse_bbob-biobj, 5)	(COMO-316_dufosse_bbob-biobj, 6)	(COMO-100_dufosse_bbob-biobj, 3)	(RM-MEDA_Auger_bbob-biobj, 1)
4	(SPEA2-platypus_Brockhoff_bbob-biobj, 5)	(COMO-316_dufosse_bbob-biobj, 4)	(NSGA-II-platypus_Brockhoff_bbob-biobj, 4)	(COMO-100_dufosse_bbob-biobj, 4)	(COMO-316_dufosse_bbob-biobj, 2)	(SPEA2-platypus_Brockhoff_bbob-biobj, 1)
5	(MO-DIRECT-HV-Rank_Wong_bbob-biobj, 4)	(SPEA2-platypus_Brockhoff_bbob-biobj, 3)	(COMO-100_dufosse_bbob-biobj, 3)	(DEMO_Tusar_bbob-biobj, 1)	(GDE3-platypus_Brockhoff_bbob-biobj, 1)	NaN

Figure 17: BBOB Bi-Objective: Ranking of Algorithms per Dimension for the Factor method, Showing (Algorithm, Number of Wins)

6.4 SUMMARY OF OBSERVATION

The initial motivation for this project was to reduce the BBOB algorithm archive to a scale where meaningful comparisons become possible. Even when restricting the analysis to a single dimension (here 20-D) and aggregating over all 24 functions, the COCO post-processing tool already produces extremely dense figures when many algorithms are included. Figure 18 illustrates this: plotting around 150 algorithms from 2009–2015 results in a cluttered ERT curve display in which individual performance profiles are unreadable, and no qualitative trend can be extracted. This makes it clear that comparing the entire archive “as-is” is not informative. This highlights how unmanageable a full 250+ algorithm comparison would be.

Using the filtering procedures developed in this project: strict-winner selection (Method 1) and factor-based near-optimality (Method 2), we reduce this overwhelming set to a small, interpretable group of consistently strong performers (Figure 19). Instead of navigating hundreds of algorithms, we isolate a concise shortlist of reliable top performers.

Across the three benchmark suites, the clarity of the selection increases. In the noiseless BBOB suite, several algorithms remain competitive. In the noisy suite, a small and stable group consistently emerges. In the bi-objective suite, dominance is even clearer, with the same algorithms leading across all dimensions and targets. Figures 18 and 19 illustrate this transformation before and after filtering.

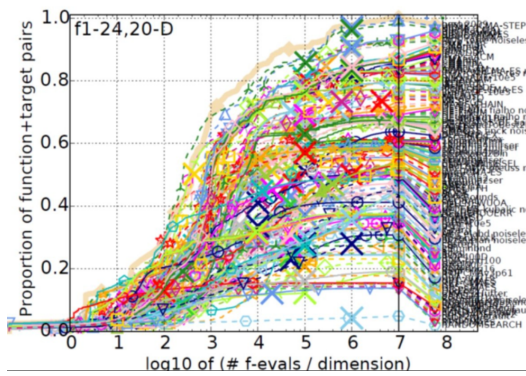


Figure 18: Years 2009-2015 Before the Filtering approach to Reduce the Number of Algorithms for the BBOB Noiseless Test Suite [3]

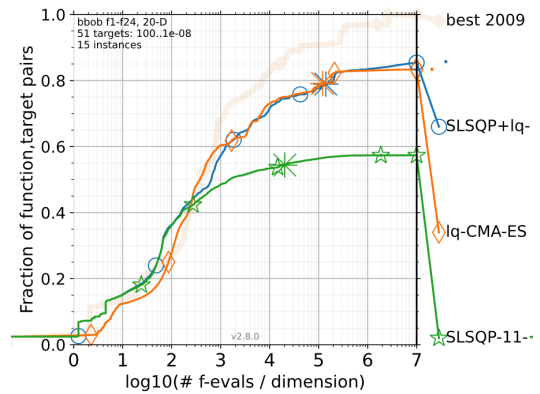


Figure 19: Top 3 Algorithms for the Noiseless 20D Test Suite after Filtering

The practical implication is straightforward. Instead of testing more than 250 algorithms, a company or practitioner can now focus on a much smaller set. Once the dimension of the problem and the required precision are known, the tables produced in this project make it easy to identify a few algorithms that are consistently reliable. This saves a substantial amount of time and avoids running large, unnecessary comparisons. Overall, the project provides a simple and practical way to narrow the search and quickly find suitable algorithms for a given black-box optimization task.

7 CONCLUSION

This project reorganized the large amount of benchmark data available in the COCO platform into a form that allows for clear and consistent comparison of algorithm performance. By relying on Expected Running Time (ERT) as the central indicator, and by analyzing results across years, dimensions, and target precisions, we established a systematic way to extract meaningful performance trends.

Two complementary selection procedures were developed: a strict-winner method that identifies the algorithm with the smallest ERT for each problem setting, and a factor-based method that highlights groups of algorithms performing within a chosen range of the best result. Together, these approaches make it possible to move from an unmanageable volume of raw COCO outputs to compact, interpretable summaries.

The application of this framework across the noiseless, noisy, and bi-objective BBOB suites reveals consistent patterns: competition among many algorithms in the noiseless case, increasing robustness and clearer separation in the noisy suite, and strong, stable dominance in the bi-objective setting. These trends become visible precisely because the data are processed in a unified manner.

Overall, the methodology developed here provides a practical tool for identifying reliable, high-performing algorithms and for understanding how their behavior evolves with dimensionality and problem difficulty. It offers a structured path to navigate the extensive COCO archives and supports more informed algorithm selection for black-box optimization tasks.

REFERENCES

- [1] N. Hansen, A. Auger, R. Ros, O. Mersmann, and M. Schoenauer. “Real-Parameter Black-Box Optimization Benchmarking: Presentation of the Noiseless Functions.” In *GECCO*, 2010.
- [2] N. Hansen, A. Auger, D. Brockhoff, and T. Tušar. Anytime performance assessment in blackbox optimization benchmarking. *IEEE Transactions on Evolutionary Computation*, 26(6):1293–1305, 2022.
- [3] COCO platform: <https://coco-platform.org/>